

2.2 Report

I implemented three bit operations using standard shift-and-mask techniques. For extracting the top-order bits (`get_top_bits`), I first compute how many positions to shift right: $shift = (\text{sizeof}(\text{unsigned int}) \times 8) - num_bits$. Shifting *value* right by *shift* moves the desired top *num_bits* down to the least significant positions. I then build a mask with *num_bits* ones, $(1 \ll num_bits) - 1$, and return `value >> shift` masked with that value. With `myaddress = 4026544704` and `NUM_TOP_BITS = 4`, this yields $1111_2 = 15$, as required. For setting and getting a bit at a specific index in a bitmap, I treat the bitmap as a byte array and do two-level addressing. I compute the target byte as $byteIndex = index / 8$ and the bit position within that byte as $bitIndex = index \% 8$. To set a bit (`set_bit_at_index`), I OR the byte with a single-bit mask: `bitmap[byteIndex] |= (1 << bitIndex)`. To read a bit (`get_bit_at_index`), I shift the byte right by *bitIndex* and AND with 1: `((bitmap[byteIndex] >> bitIndex) & 1)`. This uses little-endian bit numbering within each byte (bit 0 is the LSB). With `BITMAP_SIZE = 4` and `SET_BIT_INDEX = GET_BIT_INDEX = 17`, setting then reading returns 1, confirming the behavior.

3.3 Report

The bug was an infinite trampoline between `main` and the worker context caused by how `getcontext/setcontext` interact. `getcontext(&main_ctx)` snapshots the CPU state so that restoring `main_ctx` later resumes execution at the instruction immediately after that call with the saved stack and signal mask. We then built `worker1_ctx` with its own heap stack and `uc_link = &main_ctx`, and jumped into it with `setcontext(&worker1_ctx)`. When the worker function returned, the ucontext runtime did `setcontext(main_ctx)` under the hood (following `uc_link`), resuming `main` right after the original `getcontext`. Because the code path after that point unconditionally re-initialized `worker1_ctx` and called `setcontext` again, the program repeatedly: resume in `main` → rebuild worker → `setcontext(worker)` → worker returns → resume `main` after `getcontext` → ... This also leaked a new stack on every lap.

The fix is to make the “jump to worker” happen exactly once. I used the provided `resumed_to_main` flag (keeping all starter code intact). On first entry (`resumed_to_main == 0`), we set it to 1, create `worker1_ctx` with `uc_link = &main_ctx`, and call `setcontext(&worker1_ctx)`; by definition that call does not return on success. When the worker returns, control is transferred to `main_ctx` and execution resumes right after `getcontext`. This time `resumed_to_main == 1`, so we skip the `init/setcontext` block, print “back after worker returned via `uc_link`,” free the previously allocated stack, and terminate. Control flow is therefore: `main` → worker → `main`, exactly once, with a clean free and no loop.

Why this ensures termination: (1) there is a single dynamic allocation of the worker stack guarded by the flag; (2) `setcontext` is executed only on the first pass, so there is no path from the resumed `main` back into the worker; (3) `uc_link` is used precisely once to route the normal function return of the worker to the saved `main` context. If `setcontext` were to fail (rare), the code reports the error and exits; the success path cannot “fall through” because `setcontext` does not return. We intentionally did not switch to `swapcontext` (which would also solve it) to match the assignment’s requirement to use `setcontext`.